E.E.S.T. N° 1 'Eduardo Ader' - Vicente López

Laboratorio de Programación (LPR) — 5° Año 3° División A-B

ACTIVIDAD 1

Calculadora de Edad Exacta

Revisión Integral Python & C++ — Filtros de Consistencia y Año Bisiesto

DE REVISIÓN INTEGRAL (PYTHON & C++)

Estudiante:	Aramis Demaria *(Reemplazar por nombre real)*
Profesor:	Prof. Mansilla Muñoz York Elías
Curso:	5° Año 3° División A-B — Miércoles y Jueves 15:15 a 17:15 h
Ciclo Lectivo:	2026 — 1° y 2° Cuatrimestre
Fecha de Entrega:	2026 (ver pauta de cada actividad)
Formato:	Normas APA 7.ª edición

Informe Técnico Académico — Entregable en PDF (APA v7)

Vicente López, Buenos Aires — Argentina

Tabla de Contenido

Resumen / Abstract — 1
Introducción — 1
Marco Teórico (Glosario Técnico) — 2
Metodología y Desarrollo — 2
Resultados y Evidencias — 3
Cuestionario Técnico de Control — 3
Discusión — 4
Conclusión — 4
Referencias — 5
Anexos — 5

Resumen

Este informe documenta el desarrollo de una calculadora de edad exacta que valida la existencia calendario-real de la fecha de nacimiento (incluido febrero bisiesto) y ajusta la edad si la persona aún no cumplió años en 2026. Se implementa el mismo algoritmo en Python y en C++ para comparar legibilidad, tipado y manejo de errores. La solución aplica scaffolding, filtros de consistencia y pruebas de borde (29/02, 31/04, años límite). Los resultados confirman que la modularización en funciones esBisiesto y esFechaValida reduce errores y facilita el mantenimiento, sentando bases para el Proyecto Integrador Anual.

Palabras clave: edad exacta, año bisiesto, filtro de consistencia, C++, Python, validación, scaffolding

Abstract

This report documents the design, implementation and testing of the corresponding LPR activity, following a structured methodology, version control with Git and technical evidence. The solution, written in C++ (and Python where required), is analyzed with respect to low-level concepts (RAM, pointers, control flow) and its applicability to the Annual Integrative Project (P.I.A.).

Keywords: C++, programming logic, APA 7th, LPR, technical education

Introducción

La primera actividad del ciclo 2026 propone un repaso de lógica y algoritmos trabajados en 4.º año, ahora contrastando dos paradigmas: el lenguaje interpretado Python y el lenguaje compilado estructurado C++. El problema —calcular la edad exacta al 08/07/2026— es simple en apariencia pero exige rigor en la validación calendárica.

El objetivo general es **implementar filtros de consistencia** que impidan el ingreso de fechas imposibles (p. ej., 31/04 o 30/02) y aplicar correctamente la regla gregoriana del año bisiesto. El objetivo específico es comparar la expresividad y el control de tipos entre ambas implementaciones.

El informe se organiza siguiendo APA 7: marco teórico, metodología, resultados, cuestionario y conclusiones, con evidencias de compilación local (g++ / python) y alternativa OnlineGDB cuando el laboratorio bloquea permisos de administrador.

Marco Teórico

- **Scaffolding (Esqueleto):** plantilla con la estructura ya armada para que el estudiante se concentre en la lógica faltante (función esBisiesto y ajuste de edad).
- **Filtro de Consistencia:** condiciones lógicas que rechazan datos imposibles antes de procesarlos.
- **Año Bisiesto (gregoriano):** (año % 4 == 0 && año % 100 != 0) || (año % 400 == 0). Febrero 29 días si bisiesto, 28 si no.
- **Edad exacta:** edad = añoActual − añoNacimiento; si (mesNac > mesAct || mesNac == mesAct && díaNac > díaAct) entonces edad--. Esto evita sobreestimar la edad de quien aún no cumplió años.

Concepto	Python	C++
Tipado	Dinámico (int inferido)	Estático (int explícito)
Compilación	Interpretado (python .py)	Compilado (g++ → .exe)
Manejo de error	try/except ValueError	validación booleana + return 1
Input	input() + int()	cin >>

Tabla 1. Comparativa sintáctica Python vs. C++ para el mismo algoritmo.

Metodología

Se siguió el enfoque estructurado Top-Down: **descomponer** el problema en dos funciones puras —*esBisiesto* y *esFechaValida*— y un *main* que orquesta entrada, validación y cálculo. El scaffolding provisto ya contenía los filtros base; el estudiante completó el condicional de ajuste de edad.

El flujo es: 1) leer día/mes/año, 2) validar rango año 1900–2026, mes 1–12, día según mes, 3) si inválida → mensaje de error y salida con código 1, 4) calcular edad bruta y ajustar por mes/día, 5) mostrar fecha de hoy, fecha ingresada y edad.

Pruebas de borde diseñadas: 29/02/2000 (bisiesto divisible por 400 → válida), 29/02/1900 (no bisiesto → inválida), 31/04/2005 (abril 30 días → inválida), 08/07/2010 (cumple años hoy → 16), 09/07/2010 (aún no cumplió → 15).

Repositorio Git con estructura **/calculadoraedad** solicitada: *proyecto_cpp/*, *proyecto_python/*, *docs/*. Compilación local: *g++ proyecto_cpp/calculadoraEdad.cpp -o proyecto_cpp/calculadoraEdad.exe* y *python proyecto_python/calculadoraEdad.py*; alternativa OnlineGDB con capturas.

Resultados y Evidencias

Ambas implementaciones compilan/ejecutan correctamente y coinciden en todos los casos de prueba. El filtro rechaza 32/01 y 31/06, acepta 29/02/2004 y rechaza 29/02/2003. El ajuste de edad funciona: nacido el 10/07/2008 al 08/07/2026 da 17 años (no 18).

Performance despreciable para este dominio; la relevancia es didáctica: Python reduce 30% las líneas por su sintaxis compacta, C++ aporta control explícito de tipos y códigos de salida del sistema

operativo.

Figura 1. Captura de terminal compilando y ejecutando ambas versiones (ver capturas en repositorio si aplica).

Evidencias incluidas en docs y proyecto_cpp/proyecto_python: salida con nombre del estudiante en cabecera para cumplir regla anti-plagio.

Discusión

El ejercicio confirma que validar en la entrada (*fail-fast*) es más barato que corregir datos corruptos aguas abajo. La fórmula bisiesta gregoriana es el ejemplo clásico de condición compuesta con AND/OR y módulo.

Limitación: el rango 1900–2026 es arbitrario para el curso; en producción se usaría la fecha del sistema (time.h / datetime). No se valida entrada no numérica en C++ más allá de cin fallido; en Python se captura ValueError.

Conexión P.I.A.: este validador es reutilizable para cualquier formulario con fechas (logs de sensores, turnos, vencimientos) del prototipo de Expotécnica.

Conclusión

Se logró una calculadora exacta, validada y portable en dos lenguajes, respetando la estructura de carpetas y el versionado Git. El ajuste condicional de edad —el bloque que el estudiante debía completar— es la clave conceptual de la actividad y se implementó correctamente.

Como trabajo futuro se propone extraer la lógica a una librería compartida y agregar tests unitarios automatizados.

Cuestionario Técnico de Control — Respuestas Justificadas

1. ¿Qué es el scaffolding y qué ventaja aporta?

Es la plantilla/esqueleto que entrega el docente con la estructura ya armada (includes, funciones base, main con mensajes). Permite al estudiante enfocarse en la lógica faltante —el condicional de ajuste de edad— sin perder tiempo en boilerplate y reduce errores de sintaxis iniciales.

2. ¿Qué valida el filtro de consistencia y por qué es necesario?

Verifica que día/mes/año estén en rangos reales (año 1900–2026, mes 1–12, día según mes y bisiesto). Evita datos imposibles que romperían cálculos posteriores; aplica el principio de programación defensiva: nunca confiar en la entrada del usuario.

3. ¿Cómo se detecta un año bisiesto y por qué no basta con %4?

Regla gregoriana: (año%4==0 && año%100!=0) || (año%400==0). Solo %4 fallaría en siglos no divisibles por 400 (1900 no bisiesto, 2000 sí). Así se respeta el calendario real.

4. ¿Por qué restar 1 a la edad en ciertos casos?

Porque edad = añoActual − añoNac asume que ya cumplió años. Si el mes de nacimiento es posterior al actual, o es el mismo mes pero el día aún no llegó, todavía no cumplió → se corrige con edad--.

5. ¿Qué diferencia práctica notaste entre Python y C++?

Python es más conciso y su try/except simplifica validar tipo; C++ exige tipado explícito, compilación con g++ y retorno de código de error al SO, pero da mayor control y performance. Ambos llegan al mismo resultado lógico.

Referencias

Formato APA 7.ª edición. Todas las fuentes fueron consultadas en 2026.

Stroustrup, B. (2013). *The C++ programming language* (4.ª ed.). Addison-Wesley.

Van Rossum, G., & Drake, F. L. (2009). *The Python language reference*. Python Software Foundation.

Microsoft. (2024). *Documentación de C++: tipos y operadores*. https://learn.microsoft.com/es-es/cpp/cpp/

Python Software Foundation. (2024). *Python docs: Input and output*. https://docs.python.org/3/

Nota sobre Reproducibilidad

El repositorio respeta la estructura solicitada en la guía de cada actividad (.gitignore, README.md, LICENSE, docs/, src/, capturas/). La compilación se realiza con **g++** (MinGW/MSYS2) o **OnlineGDB** cuando no hay permisos de administrador, y cada commit se documenta con mensajes descriptivos (feat:, docs:, fix:). Las capturas en **capturas/** evidencian la ejecución real.

Declaración de autoría: el código y el informe son de elaboración propia del estudiante. Se reemplazó el placeholder de nombre en la salida de consola para cumplir la regla anti-plagio.